

Week 13

Algorithmic
optimization
and AI agents

Nebius Academy



A math case of
AlphaEvolve

Bruhat graph for S_n

Every $\tau \in S_n$ is a product of $(i, i + 1)$:

$$\tau = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \end{pmatrix} = (1,2)(2,3)$$

Minimal length: $l(\tau) = 2$

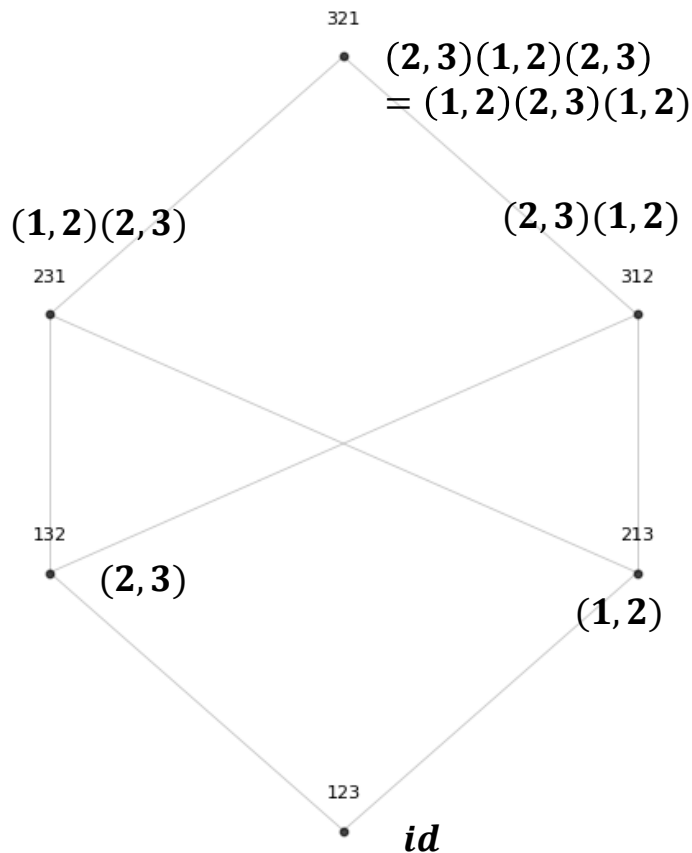
Bruhat graph:

- Vertices: $\tau \in S_n$,
- Edges: $\sigma - \tau$ if $\sigma = (i, i + 1)\tau$ or $\sigma = \tau \cdot (i, i + 1)$ and $l(\sigma) = l(\tau) \pm 1$

Bruhat intervals that are large hypercubes

<https://arxiv.org/abs/2601.01235>

Bruhat cover graph of S_3



The question

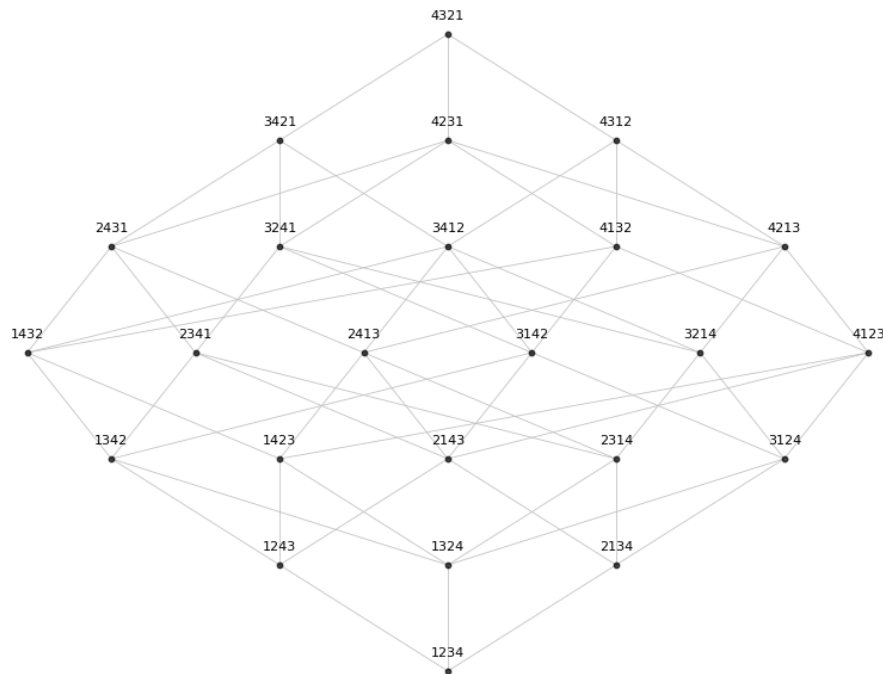
A hypercube:

- Vertices = subsets of a m -element set
- Edges = inclusion relation

An **interval**: vertices that are *between* two given vertices

If $n = 2^m$, what is the largest interval in the graph which is a hypercube?

Bruhat cover graph of S_4



Bruhat intervals that are large hypercubes

<https://arxiv.org/abs/2601.01235>

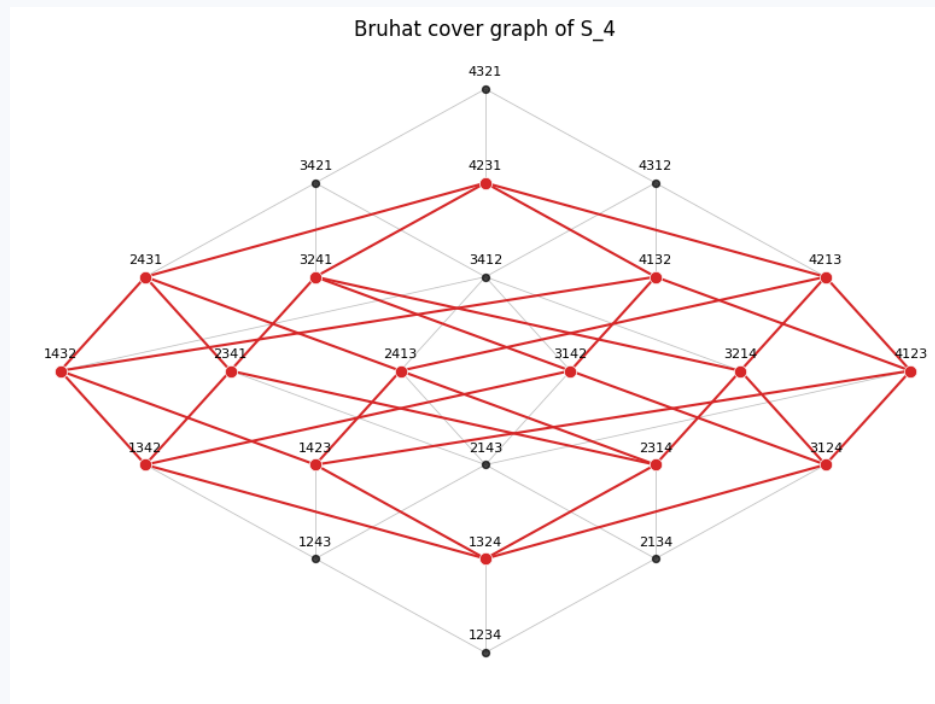
The question

A hypercube:

- Vertices = subsets of a m -element set
- Edges = inclusion relation

An **interval**: vertices that are *between* two given vertices

If $n = 2^m$, what is the largest interval in the graph which is a hypercube?



Bruhat intervals that are large hypercubes

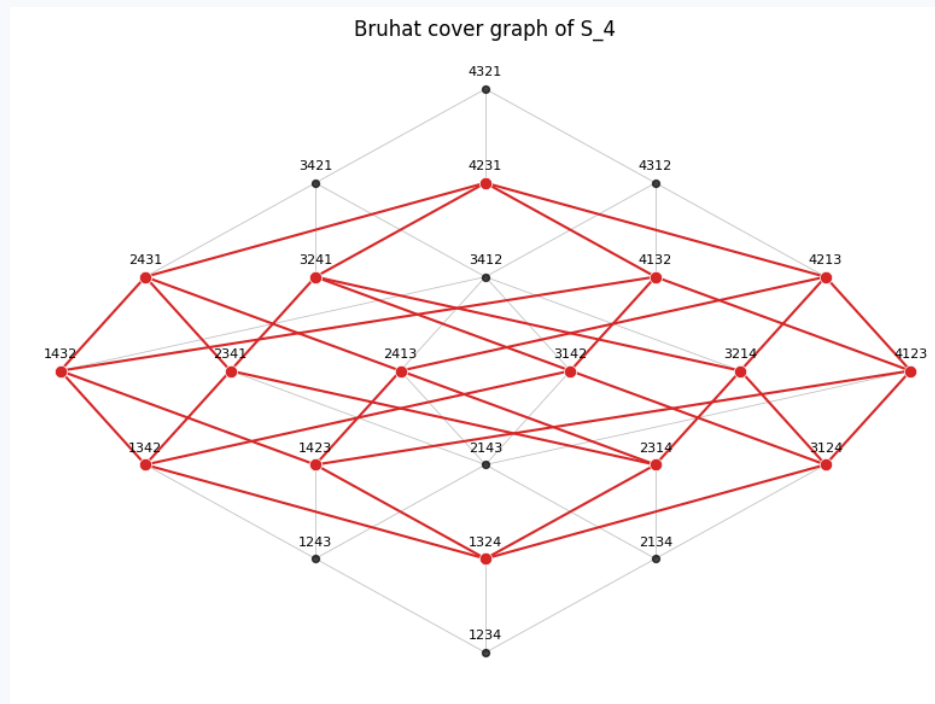
<https://arxiv.org/abs/2601.01235>

The unexpected findings

The **actual task** was:

- For each n , output a pair of permutations $x, y \in S_n$ with $d(x, y)$ as large as possible, where $d(x, y)$ is the Kazhdan–Lusztig d -invariant.

But they actually found intervals in Bruhat graphs for $n = 2^m$, which are hypercubes of dimension $m2^{m-1}$.



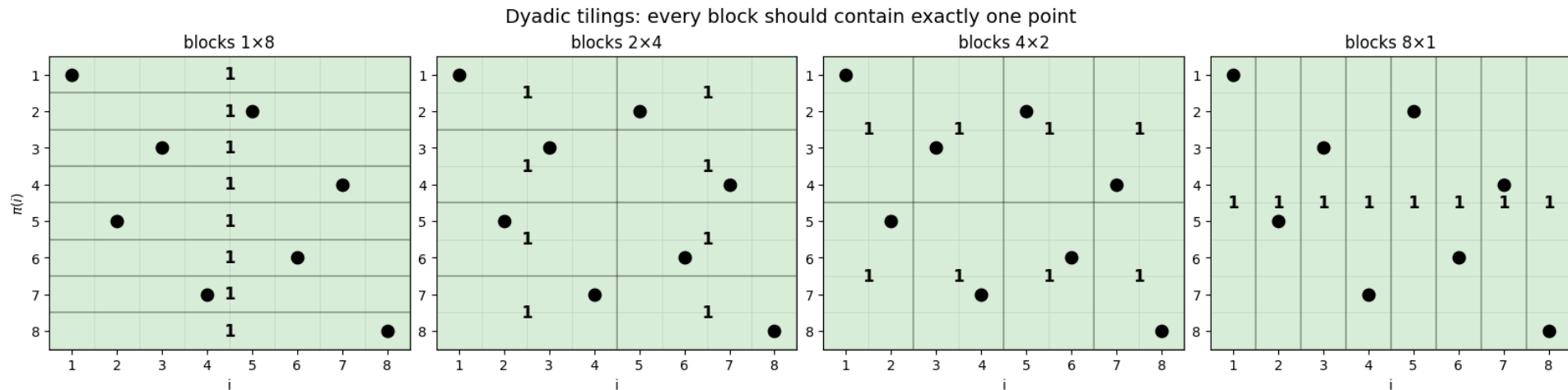
Bruhat intervals that are large hypercubes

<https://arxiv.org/abs/2601.01235>

Interpretation

The said interval consist of **dyadically well-distributed permutations**: permutation matrices whose 1's are perfectly balanced across all dyadic block decompositions.

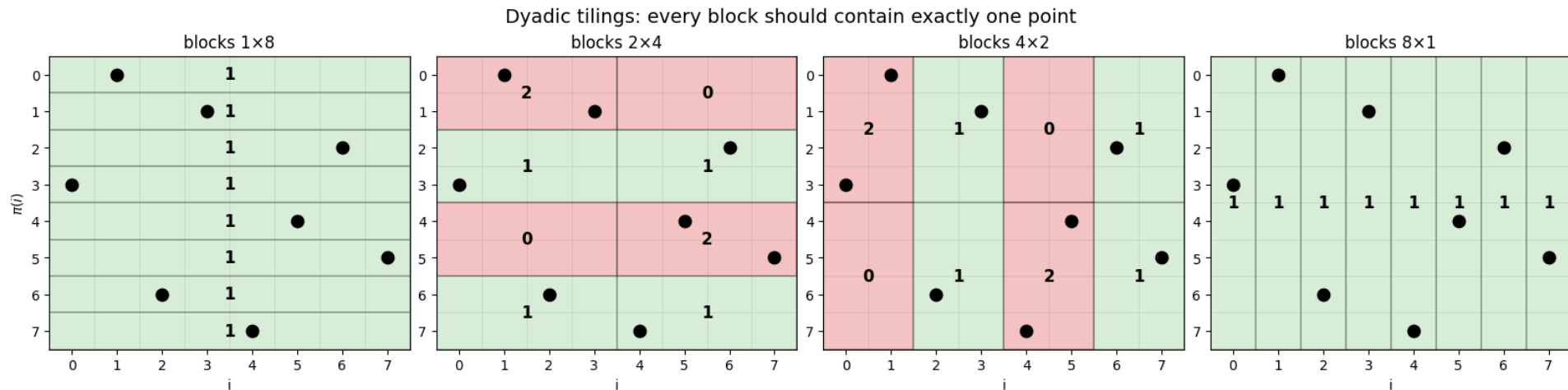
Which means: if we cut the $2^m \times 2^m$ permutation matrix into tiles of size $2^k \times 2^{m-k}$, each of them will contain only one 1.



Interpretation

The said interval consist of **dyadically well-distributed permutations**: permutation matrices whose 1's are perfectly balanced across all dyadic block decompositions.

Which means: if we cut the $2^m \times 2^m$ permutation matrix into tiles of size $2^k \times 2^{m-k}$, each of them will contain only one 1.



Searching for
algorithms -> searching
for search strategies

From searching for code to searching for search strategy

Example task: find the largest constant $\mathcal{C}$ such that

$$\max_{-\frac{1}{2} \leq t \leq \frac{1}{2}} \int_{\mathbb{R}} f(x-t)f(x)dx \geq \mathcal{C} \left(\int_{-\frac{1}{4}}^{\frac{1}{4}} f(x) dx \right)^2$$

holds for every non-negative f .

Researchers from DeepMind were able to improve to $\mathcal{C} \leq 1.5032$.

From searching for code to searching for search strategy

Example task: find the largest constant $\mathcal{C}$ such that

$$\frac{\max_{-\frac{1}{2} \leq t \leq \frac{1}{2}} \int_{\mathbb{R}} f(x-t)f(x)dx}{\left(\int_{-\frac{1}{4}}^{\frac{1}{4}} f(x) dx\right)^2} \geq \mathcal{C}$$

holds for every non-negative f .

Researchers from DeepMind were able to improve to $\mathcal{C} \leq 1.5032$.

Frome searching for code to searching for search strategy

$$\frac{\max_{-\frac{1}{2} \leq t \leq \frac{1}{2}} \int_{\mathbb{R}} f(x-t)f(x)dx}{\left(\int_{-\frac{1}{4}}^{\frac{1}{4}} f(x) dx\right)^2} \rightarrow \min_f$$

From searching for code to searching for search strategy

$$\frac{\max_{-\frac{1}{2} \leq t \leq \frac{1}{2}} \int_{\mathbb{R}} f(x-t)f(x)dx}{\left(\int_{-\frac{1}{4}}^{\frac{1}{4}} f(x) dx\right)^2} \rightarrow \min_f$$

Search space is step functions: $f(x) = a_1 \mathbb{I}[I_1](x) + \dots + a_n \mathbb{I}[I_n](x)$, where $a_i \geq 0$ and $I_1, \dots, I_n$ are uniform intervals covering $\left[-\frac{1}{4}, \frac{1}{4}\right]$.

Your task is to write a search function that searches for the best sequence of coefficients. Your function will have 1000 seconds to run, and after that it has to have returned the best sequence it found. If after 1000 seconds it has not returned anything, it will be terminated with negative infinity points. All numbers in your sequence have to be positive or zero.

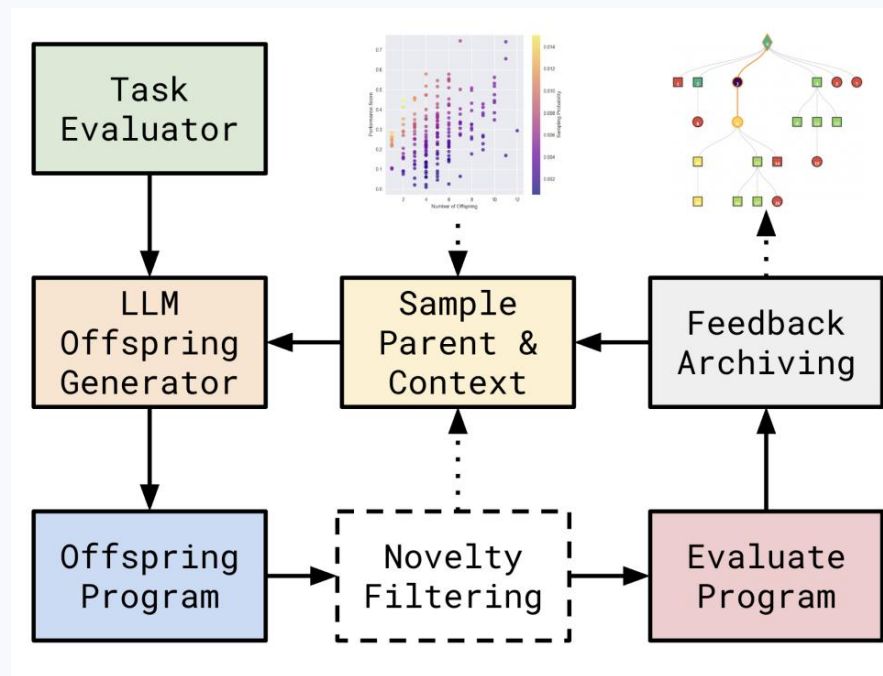
ShinkaEvolve

An open-source analog
of AlphaEvolve

ShinkaEvolve

Input

- Problem description prompt
- Checker: necessary conditions + score (what we optimize)
- Initial code



ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution

<https://arxiv.org/abs/2509.19349>

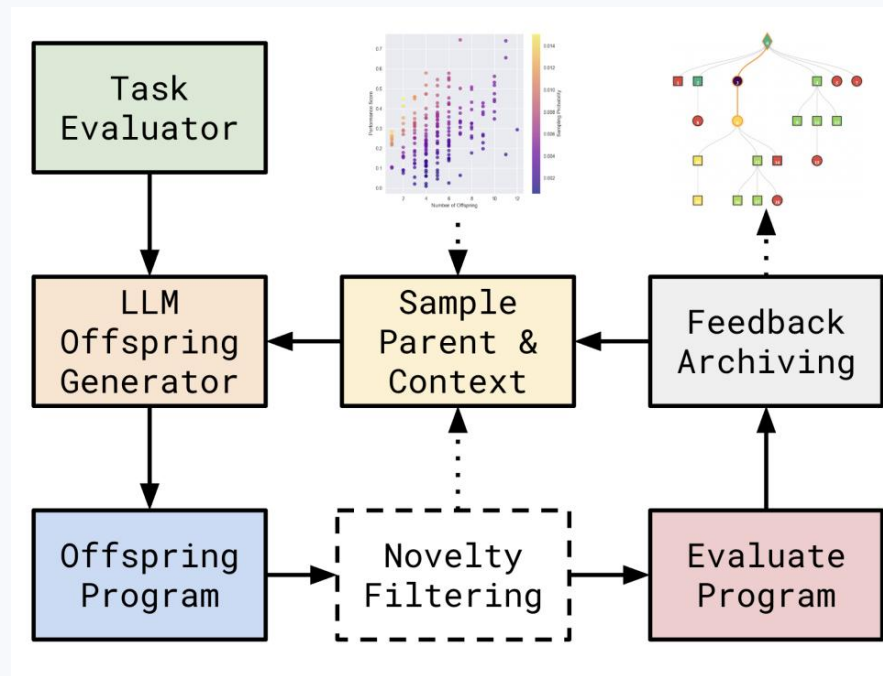
ShinkaEvolve

Keep a fixed-size library of programs.

➤ **Step 1: Select the “parent” program to be improved**

Weighted sampling strategy balancing:

- Performance: How good is the program's score
- Novelty: How many times has this program already been used as a parent? (Fewer is better.)



ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution

<https://arxiv.org/abs/2509.19349>

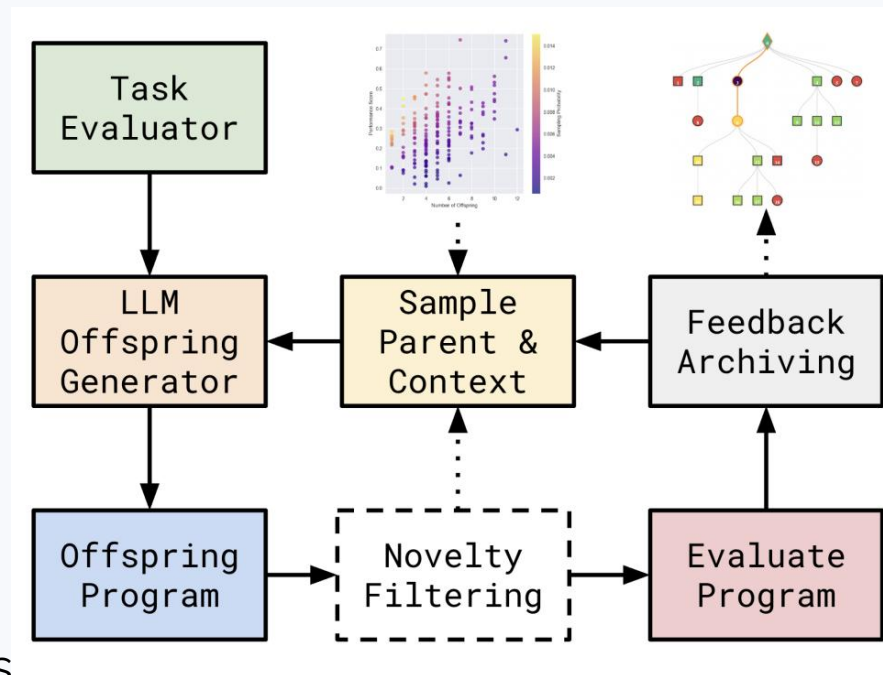
ShinkaEvolve

> Step 1: Select the “parent” program to be improved

> **Step 2: Assemble the LLM prompt**

It includes:

- The “parent” program
- Inspiration programs
- The goal
- The fitness score of the “parent” program and textual feedback (if any) that the judge provided with the fitness score.

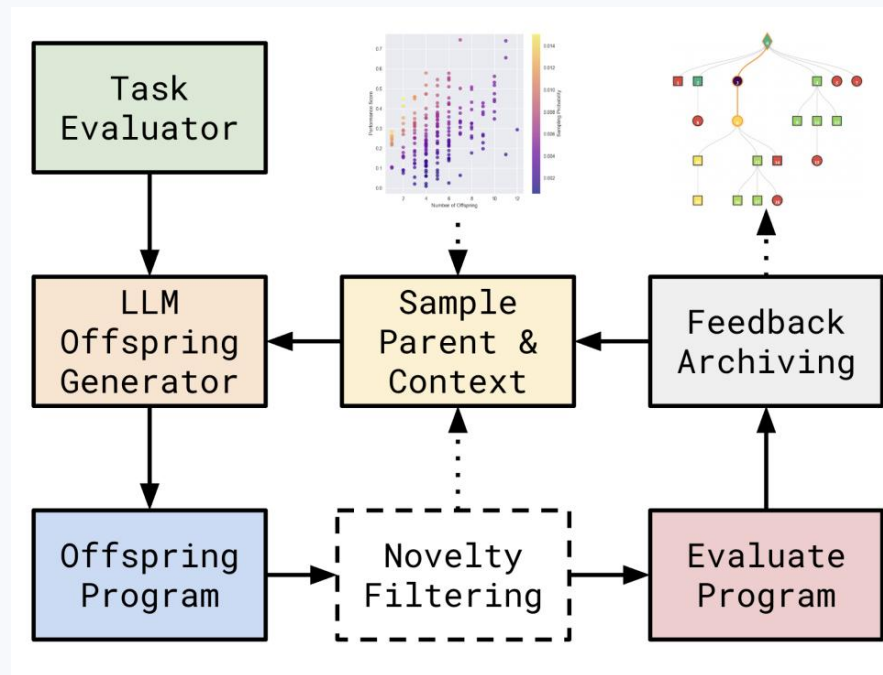


ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution

<https://arxiv.org/abs/2509.19349>

ShinkaEvolve

- > Step 1: Select the “parent” program to be improved
- > Step 2: Step 2: Assemble the LLM prompt
- > **Step 3: Mutate the “parent” program with an LLM, producing a new “offspring” program**



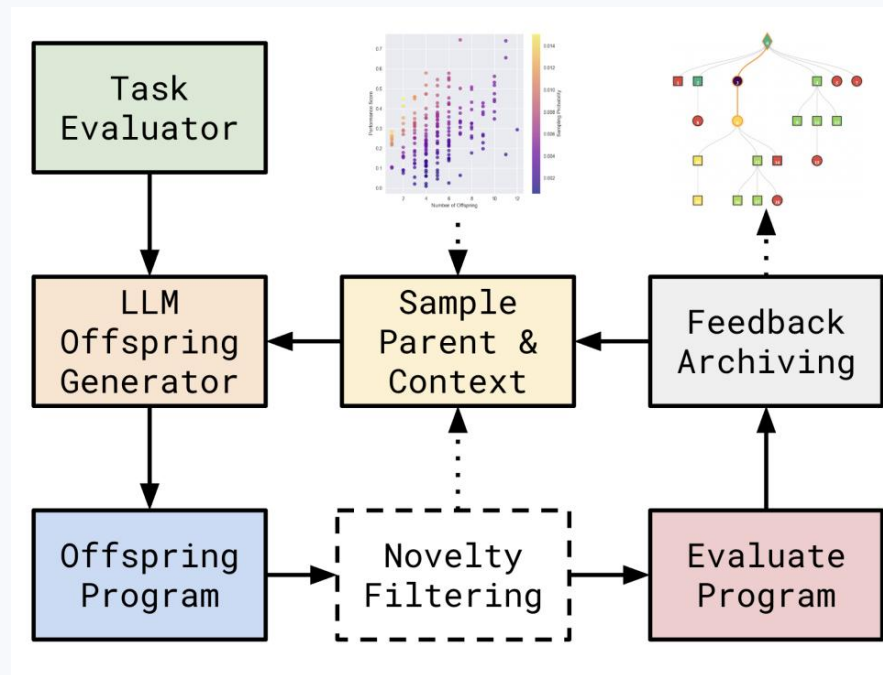
ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution

<https://arxiv.org/abs/2509.19349>

ShinkaEvolve

- > Step 1: Select the “parent” program to be improved
- > Step 2: Assemble the LLM prompt
- > Step 3: Mutate the “parent” program with an LLM, producing a new “offspring” program
- > **Step 4: Filter the “offspring” for novelty based on:**

- Embedding similarity
- LLM-as-a-Novelty-Judge



ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution

<https://arxiv.org/abs/2509.19349>

- | | | | |
|--|---------|-----------|---------|
| | Program | Filtering | Program |
|--|---------|-----------|---------|



ShinkaEvolve

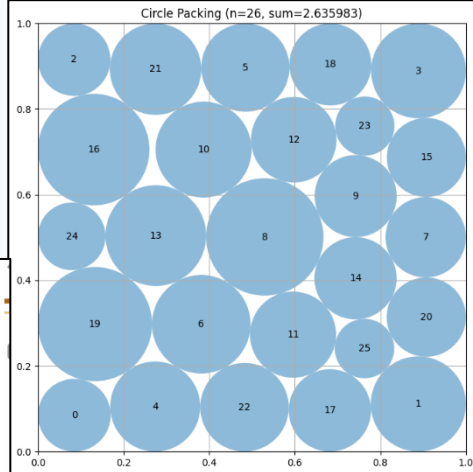
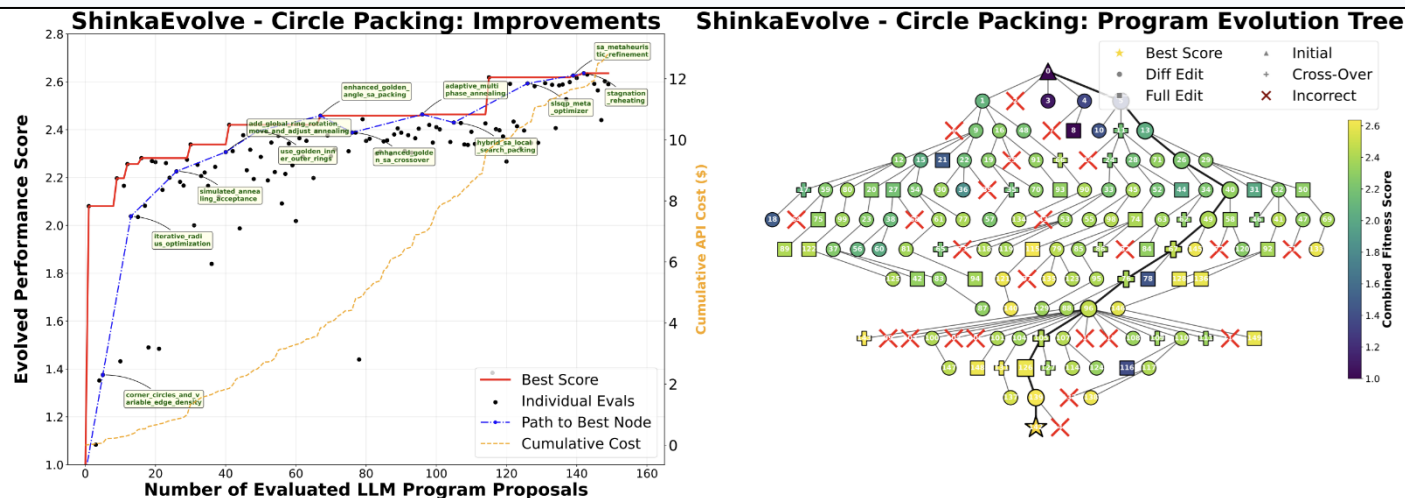


Figure 5 | **SHINKAEVOLVE on Circle Packing Task.** *Left:* SHINKAEVOLVE outperforms AlphaEvolve’s solution within less than 150 program evaluations. *Right:* SHINKAEVOLVE’s program evolution tree demonstrates the iterative composition of stepping stones into high-performing solutions.

ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution

<https://arxiv.org/abs/2509.19349>

Code time!

Further topics around
AlphaEvolve

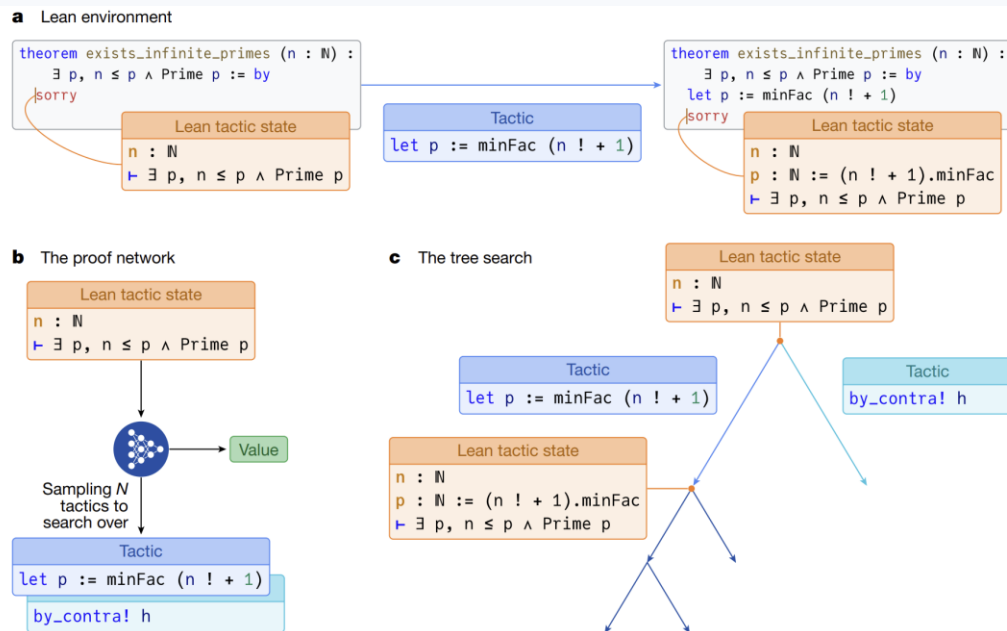
A workflow – AlphaProof

Tree search, where

- Action is a Lean step
- Scoring is made with

$$Q(s, a) + c(s)\pi^{1/\tau}(a|s) \frac{\sqrt{\sum_b N(s, b)}}{N(s, a) + 1}.$$

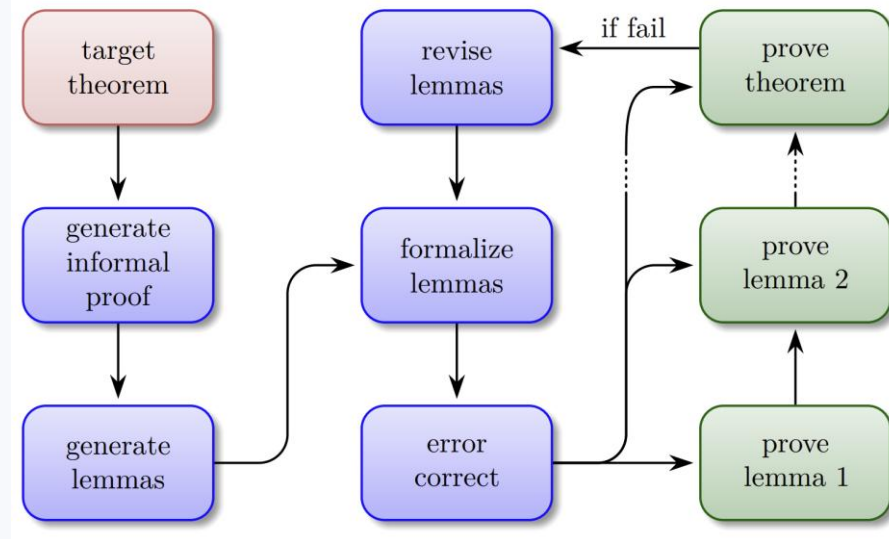
$\pi(s, a)$ = how likely the LLM thought it was
 $N(s, a)$ = times this action was tried
 $c(s)$ - exploration factor



A workflow – Aristotle

Aristotle is an agentic system for theorem proving

Recovers **sorry** statements in Lean code

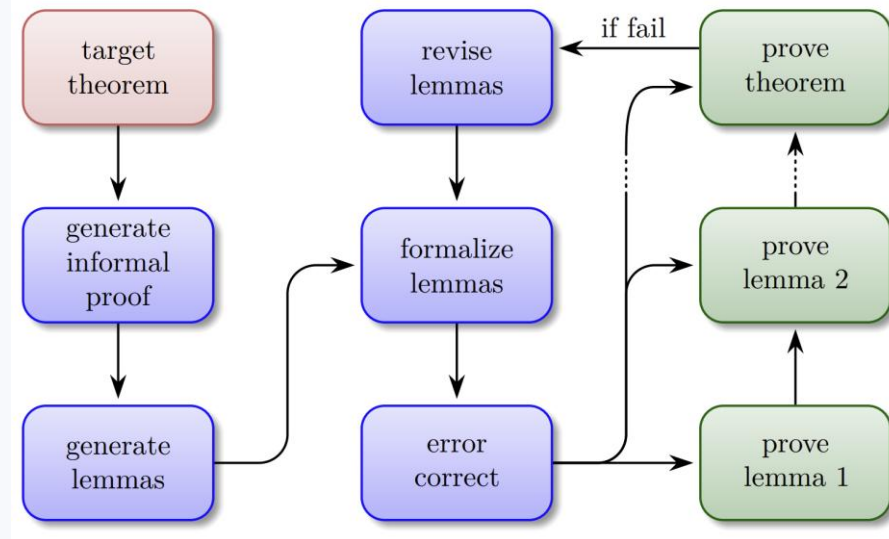


A workflow – Aristotle

Recovers **sorry** statements in Lean code

Pipeline:

- Generate informal proof
- Split it into lemmas
- Formalize these lemmas
- Prove each of them in Lean



A workflow – Aristotle

Recovers **sorry** statements in Lean code

Lemma level: performs MCTS

Actions = lean steps

Score balances:

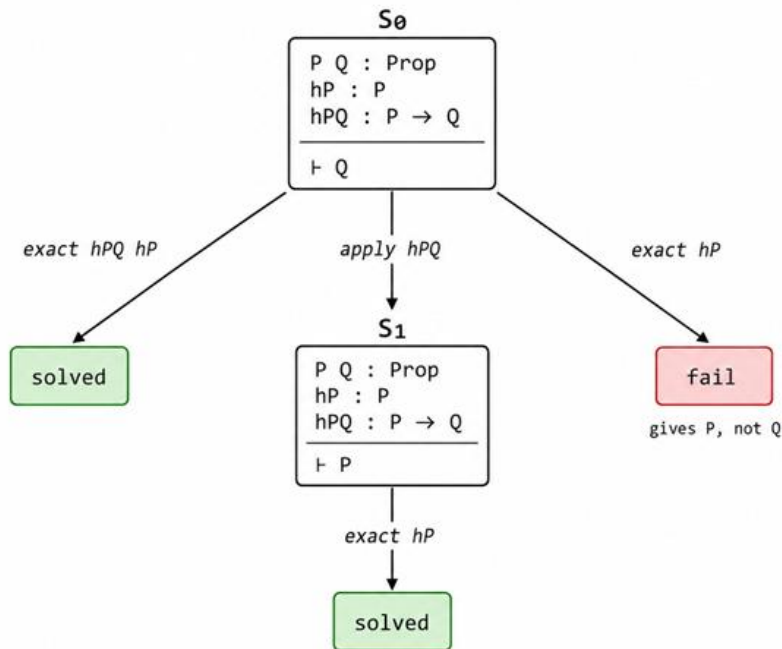
- Exploitation – how promising the current proof is (measured by a trained **value function**)
- Exploration: $\frac{P(s,a)\sqrt{N(s)}}{1 + N(s,a)}$

$P(s,a)$ = how likely the LLM thought it was

$N(s)$ = visits to the current proof state

$N(s,a)$ = times this action was tried

```
theorem toy (P Q : Prop) (hP : P) (hPQ : P → Q) : Q := by
```



Aristotle: IMO-level Automated Theorem Proving
<https://arxiv.org/abs/2510.01346>

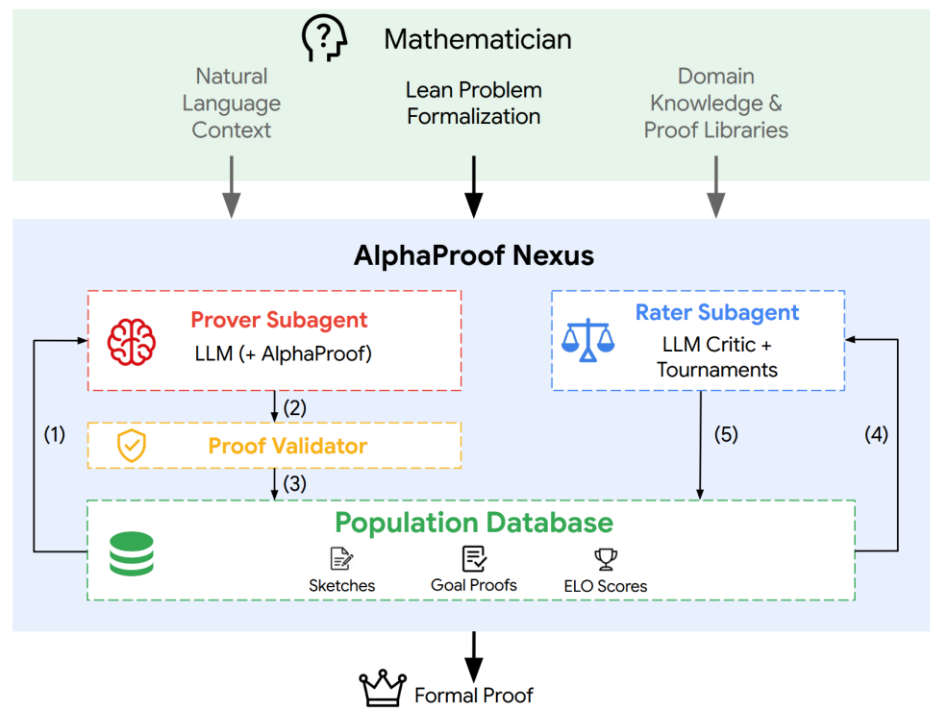
A workflow – AlphaProof Nexus

Proves theorems in Lean

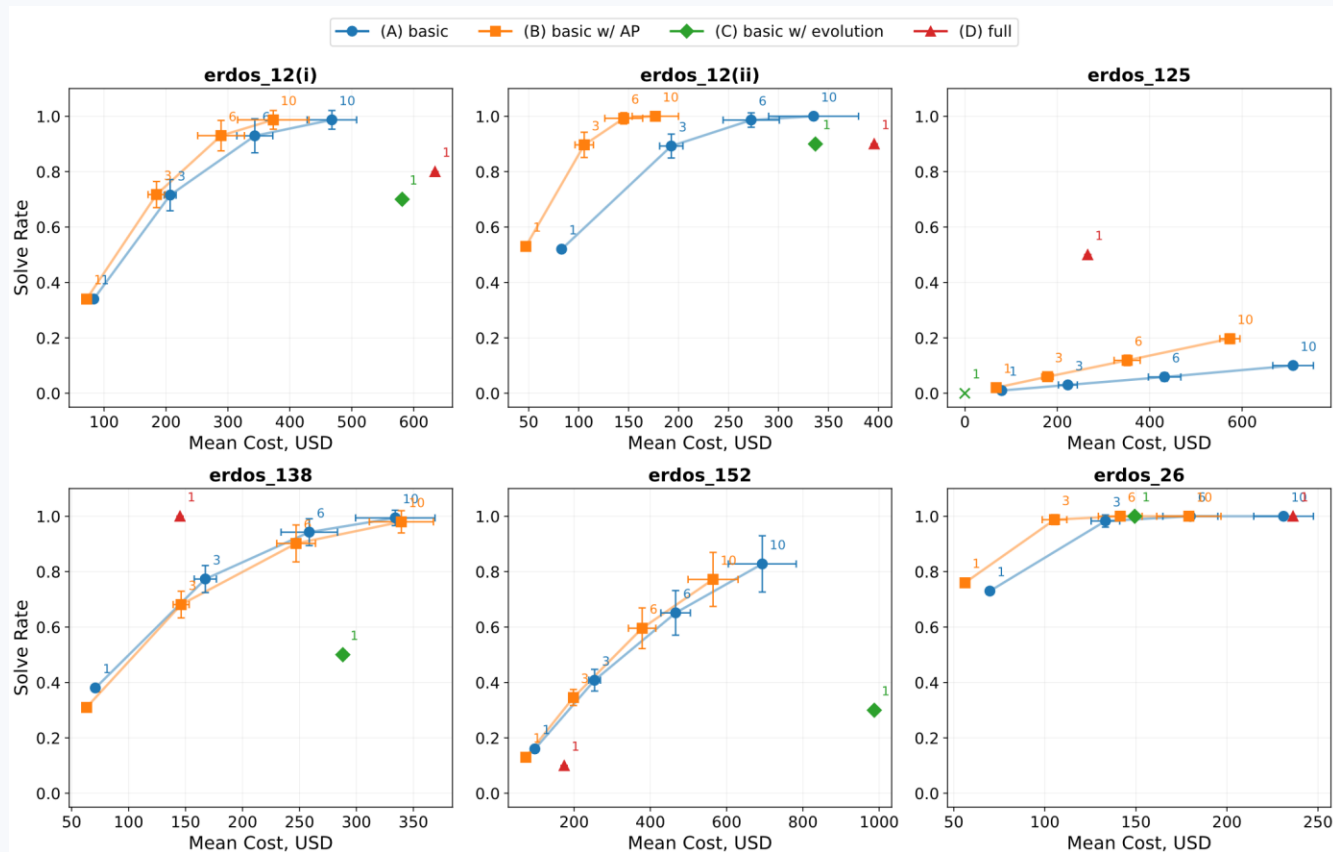
Works with the population of proof sketches.

Run many asynchronous **prover subagents**:

1. Sample a parent sketch from the population.
2. Build a prompt using that sketch, prior attempts AlphaProof feedback, etc.
3. Let the **prover LLM** edit the sketch.
4. Let it optionally call **AlphaProof** on subgoals.
5. **Validate** the resulting sketch.
6. If complete, **return final proof**.
7. If partial but valid, **add it to the population database**.



AlphaProof Nexus – effect vs cost



Results

- Proved **9 out of 353** formalized Erdős-problem statements
- Proved **44 of 492** autoformalized open OEIS conjectures.
- Proved an exact $O\left(\frac{1}{t}\right)$ last-iterate convergence rate for Anchored Gradient Descent-Ascent.
- Proved log-concavity of pure O-sequences of codimension 3 and type 2.
- Proved a structured bipartite graph reconstruction theorem and one of the Graffiti / Written on the Wall conjectures.
- Helped resolve Ben Green's problem #57 by proving a formal counterexample over $\mathbb{Z}/3\mathbb{Z}$.
- Resolved several conjectures about monochromatic quantum graphs, including no-solution results for certain N, d regimes.

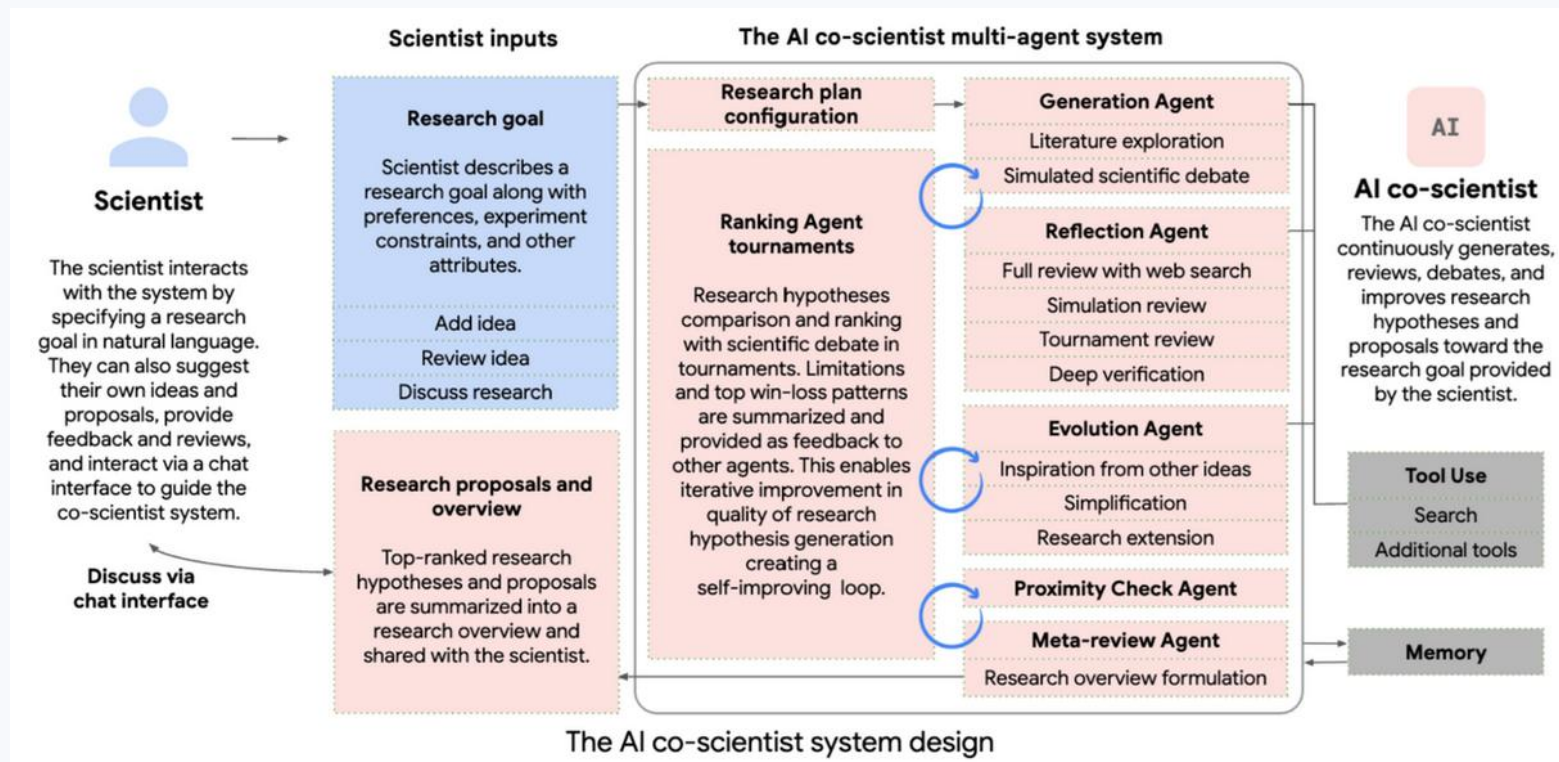
The graph conjecture

$$L(G) \geq 2(l(G) - 1),$$

where

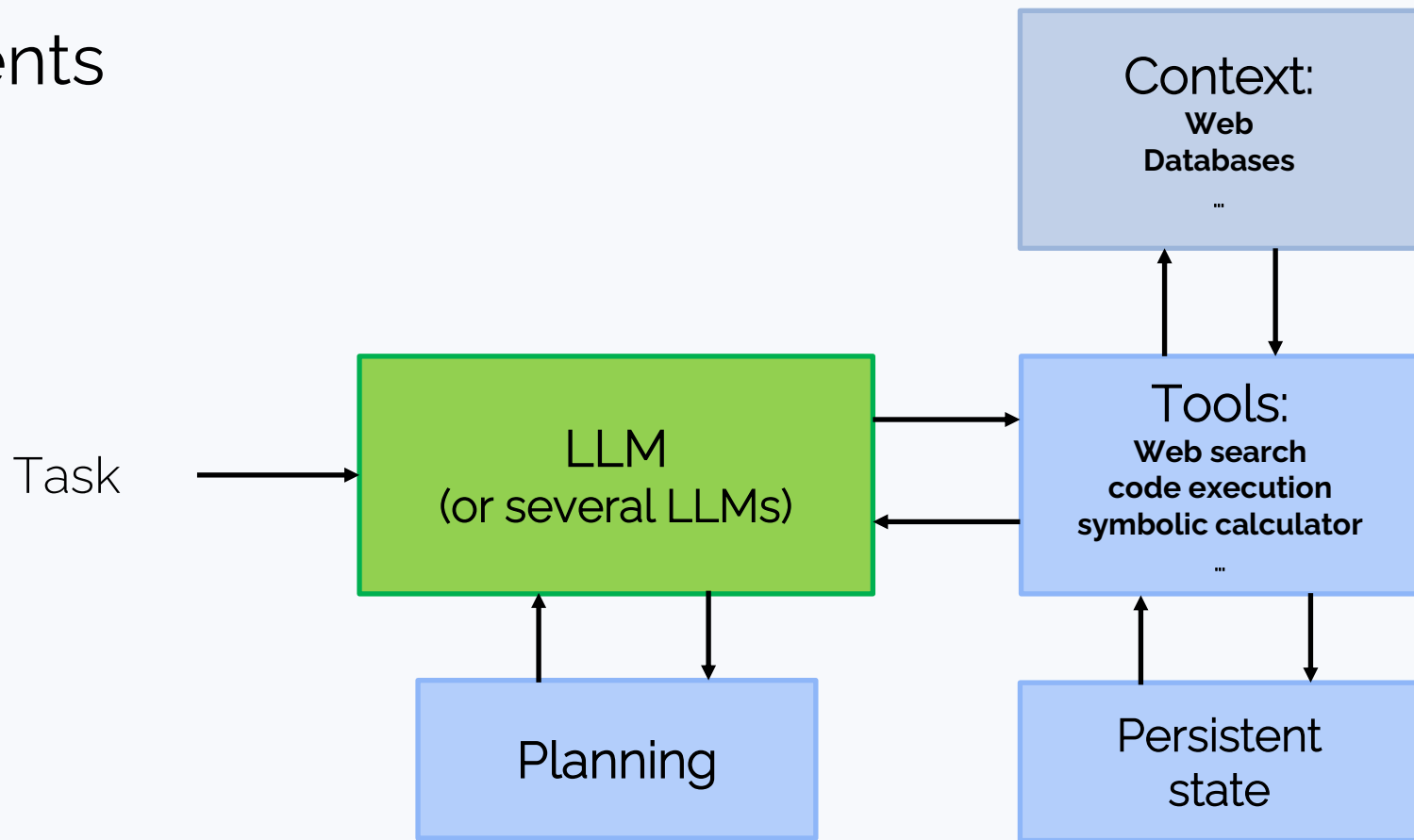
- $L(G)$ is the maximal number of leaves in a spanning tree of G
- $l(G)$ is the average $\alpha(N(v))$, where $N(v)$ is the set of vertices adjacent to v and α is the independence number

A huge agentic system: Google's AI co-scientist



AI agents

Agents



Why would you need an agent of your own? (And not just Claude Code or Chat GPT)

- Repetitive workflows
- Persistent states and databases
- Using different models

Why would you need an agent of your own? (And not just Claude Code or Chat GPT)

- Repetitive workflows
- Persistent states and databases
- Using different models

Examples:

- Scoring homeworks
- A paper explorer workflow + chatbot

Tool calling

Tool calling basics

- A **tool** is any function `function(parameters)`
- It's description is baked into a prompt
- An LLM generates a JSON string when it's time to call a tool
- This JSON is captured and executed by an external system

```
{  
  "type": "function_call",  
  "id": "fc_123",  
  "call_id": "call_abc123",  
  "name": "get_current_weather",  
  "arguments": "{\"location\":\"London, UK\",\"units\":\"metric\"}"  
}
```

```
{  
  "type": "function",  
  "name": "get_current_weather",  
  "description": "Get the current weather for a city.",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "location": {  
        "type": "string",  
        "description": "City and country, e.g. London, UK"  
      },  
      "units": {  
        "type": "string",  
        "enum": ["metric", "imperial"]  
      }  
    },  
    "required": ["location", "units"],  
    "additionalProperties": false  
  }  
}
```

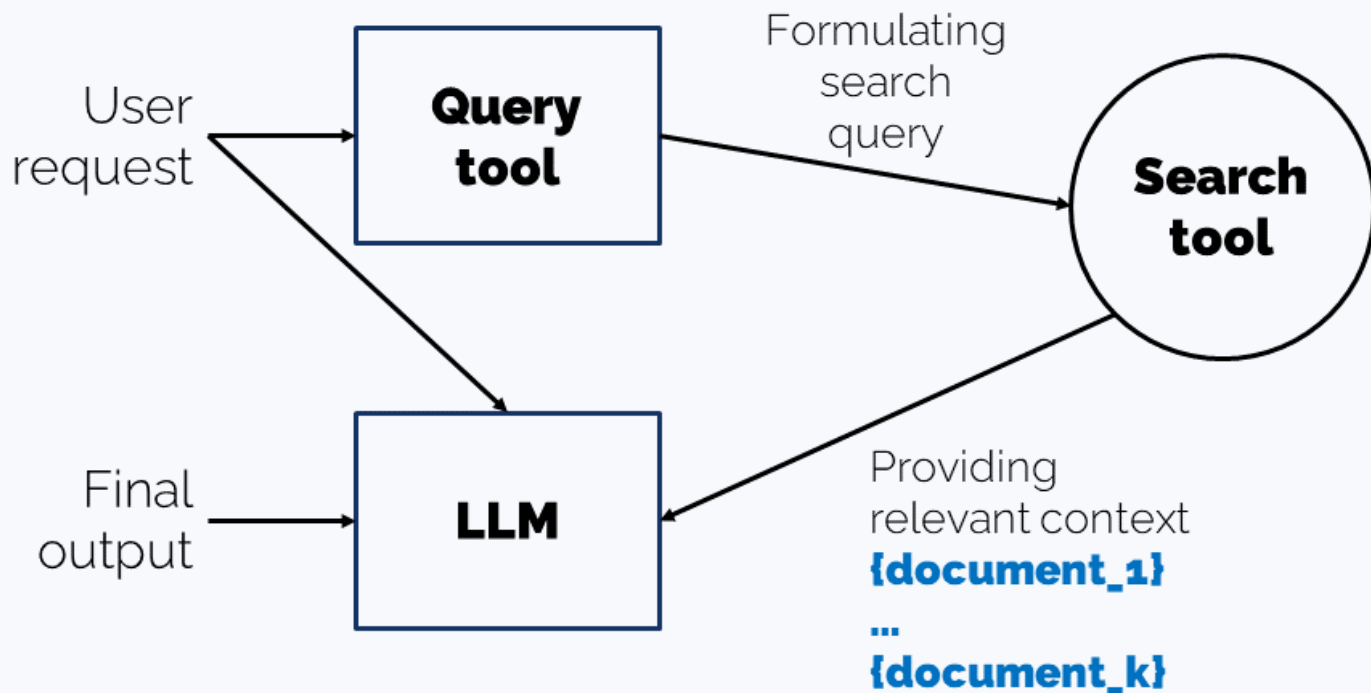
RAG

Databases aren't only for search!

We also save lots of different stuff, which can be used by us or by the agent.

- For the hometask checker agent: good to log every act of checking (date, submission, grade, feedback)
- For the paperwatch agent: papers, links, user's notes etc

Retrieval Augmented Generation (RAG)



Search options for AI agents

- Web Search (**Tavily**, Arxiv API,...)
- Local database; search by
 - Full text / keywords (“Lean”, “co-scientist”)
 - Tags (“#rl-posttraining”)
 - Structured queries (SQL-style: “Fetch me the email of every student who got bad grades at hometask 3”)
 - Fuzzy semantics (“Something like AlphaEvolve, but not exactly AlphaEvolve”)

Obsidian vaults

An **Obsidian vault** is a folder with markdown files.

Pages contain:

- Markdown
- Links ("This is related to [[DAPO]] and [[GRPO]].")
- Tags (#rl-posttraining #ai-for-math #course-material)
- YAML frontmatter / properties

```
MyResearchVault/  
Papers/  
  AI for Math/  
    2026-05-paper-title.md
```

```
LLM Reasoning/  
  dapo.md  
  grpo.md
```

```
Concepts/  
  Reinforcement Learning.md  
  Lean formalisation.md
```

```
Weekly Reviews/  
  2026-W21.md
```

```
_agent/  
  preferences.md  
  search_profiles.yaml  
  judgement_rules.md
```

Obsidian vaults

An **Obsidian vault** is a folder with markdown files.

Pages contain:

- Markdown
- Links ("This is related to [[DAPO]] and [[GRPO]].")
- Tags (#rl-posttraining #ai-for-math #course-material)
- YAML frontmatter / properties

type: paper

title: "Steering LLMs with the Dark side of the Force"

authors:

- Sheev Palpatine
- Anakin Skywalker

date: 2026-05-20

topics:

- rl-posttraining
- llm-reasoning

status: unread

interestingness: high

respectability: medium

code_available: false

recommendation: skim

Vector search



X

X



X



X

X

X



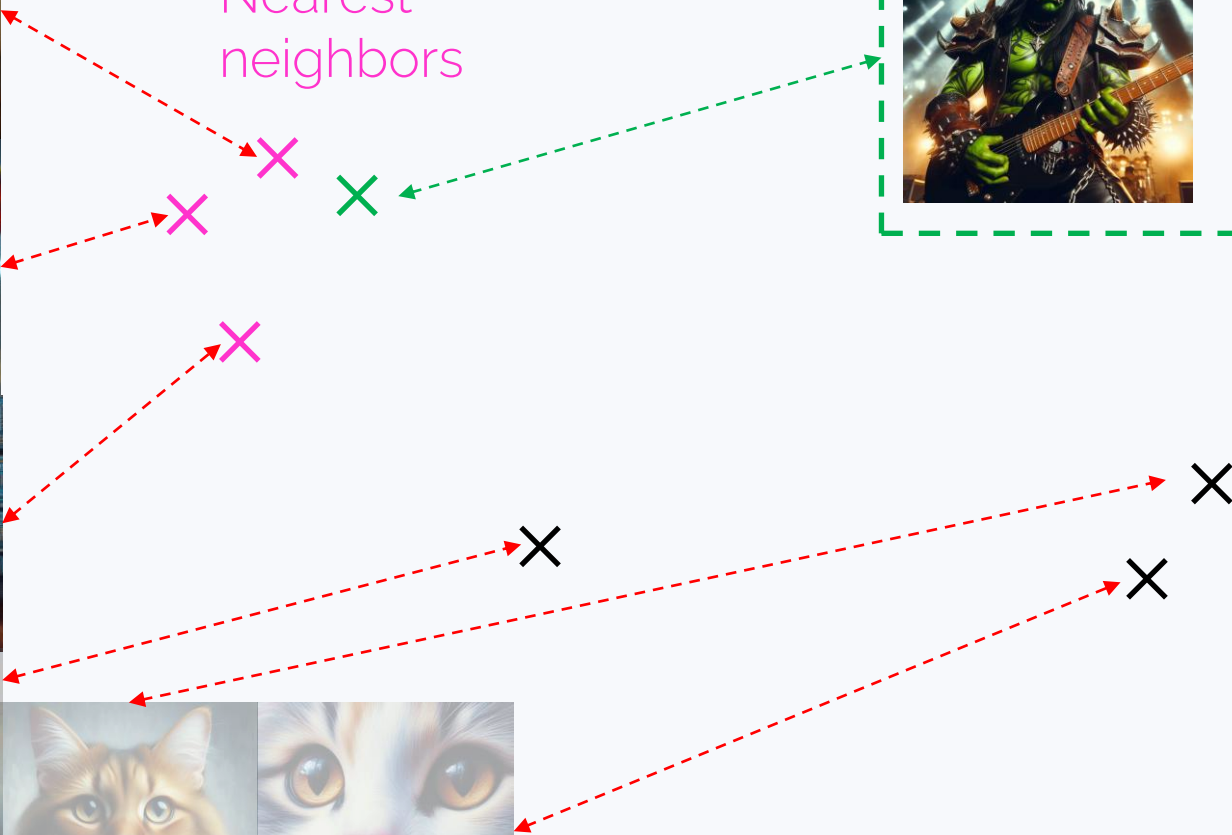
Similarity in meaning
 $\approx$ geometric distance



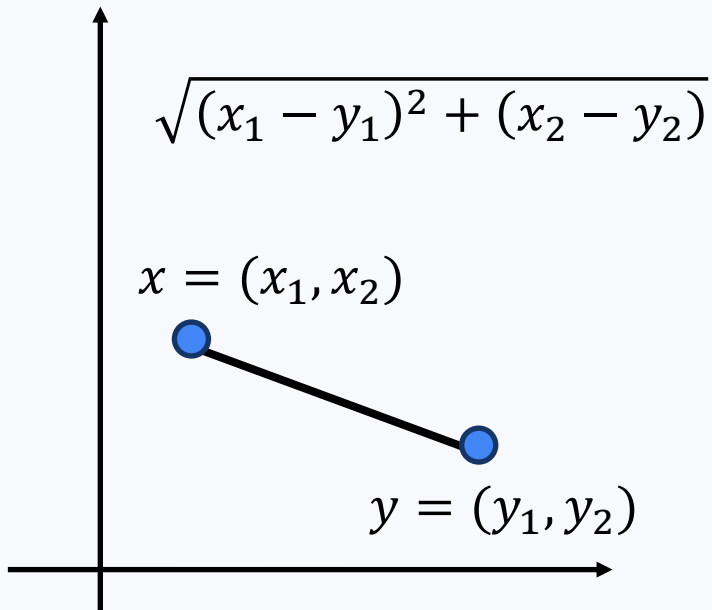
Nearest
neighbors



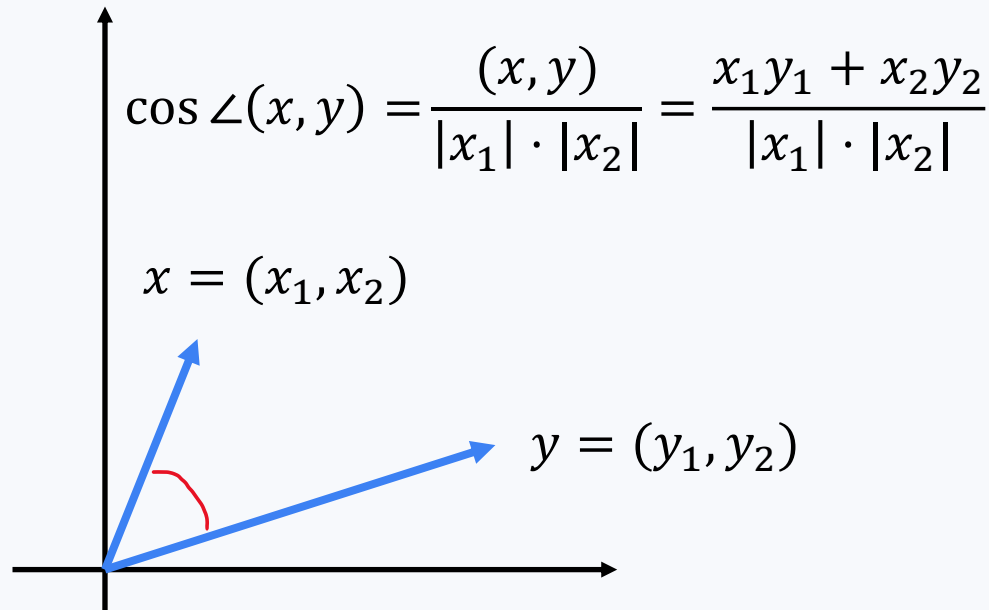
Query



Euclidean
distance



Cosine
distance



Sometimes just (x, y) is used

Predicting entailment/contradiction

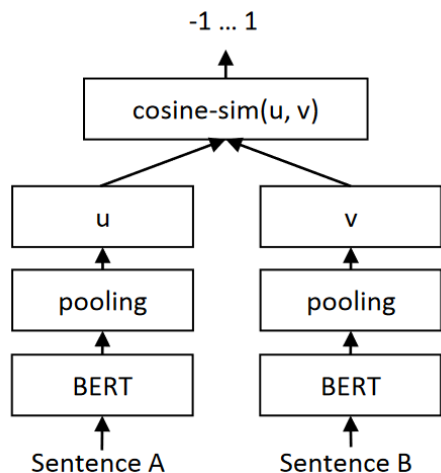
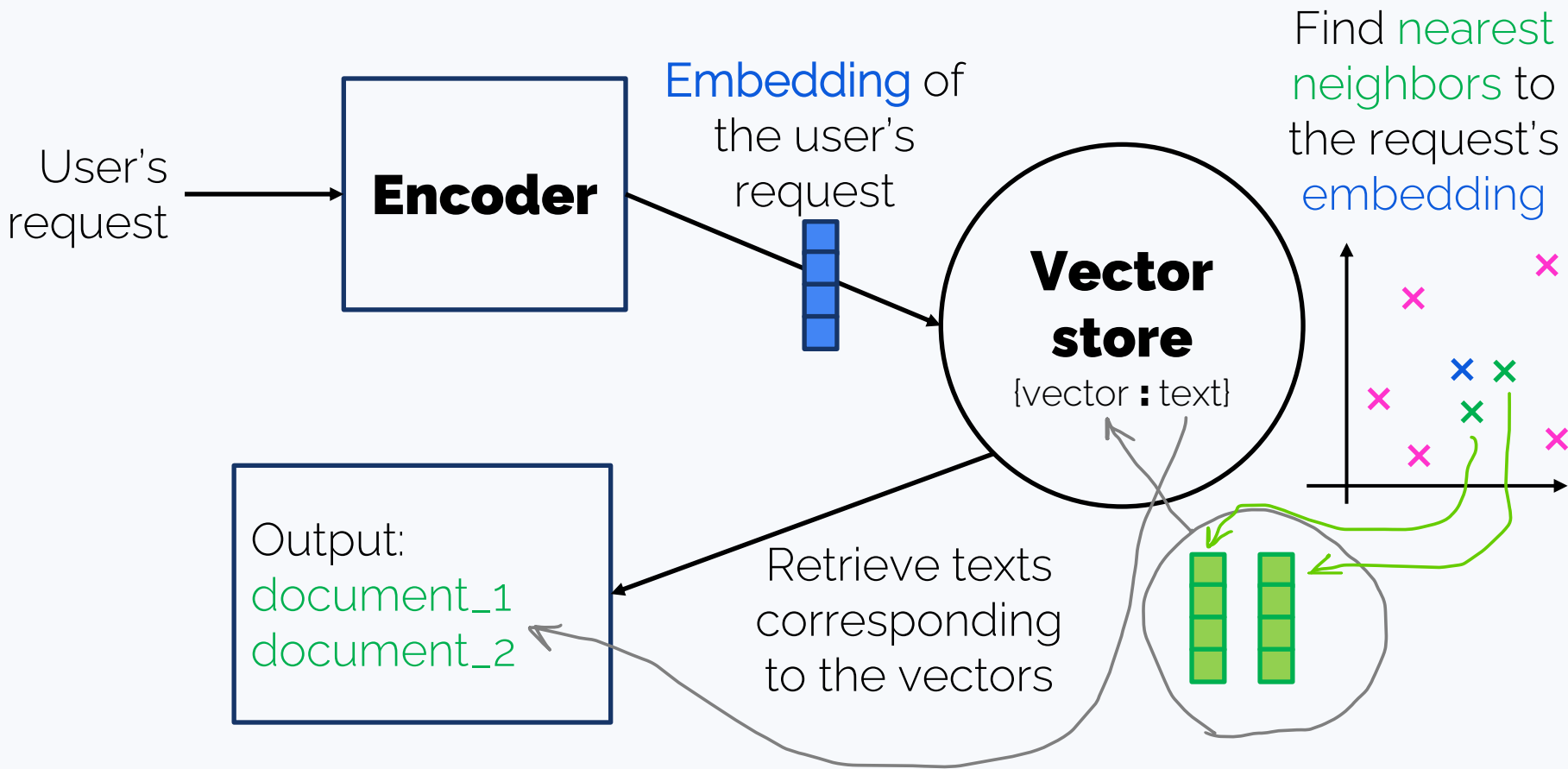


Figure 2: SBERT architecture at inference, for example, to compute similarity scores. This architecture is also used with the regression objective function.

A man inspects the uniform of a figure in some East Asian country.	contradiction C C C C C	The man is sleeping
An older and younger man smiling.	neutral N N E N N	Two men are smiling and laughing at the cats playing on the floor.
A black race car starts up in front of a crowd of people.	contradiction C C C C C	A man is driving down a lonely road.
A soccer game with multiple males playing.	entailment E E E E E	Some men are playing a sport.
A smiling costumed woman is holding an umbrella.	neutral N N E C N	A happy woman in a fairy costume holds an umbrella.

Table 1: Randomly chosen examples from the development section of our new corpus, shown with both the selected gold labels and the full set of labels (abbreviated) from the individual annotators, including (in the first position) the label used by the initial author of the pair.

RAG (retrieval augmented generation)

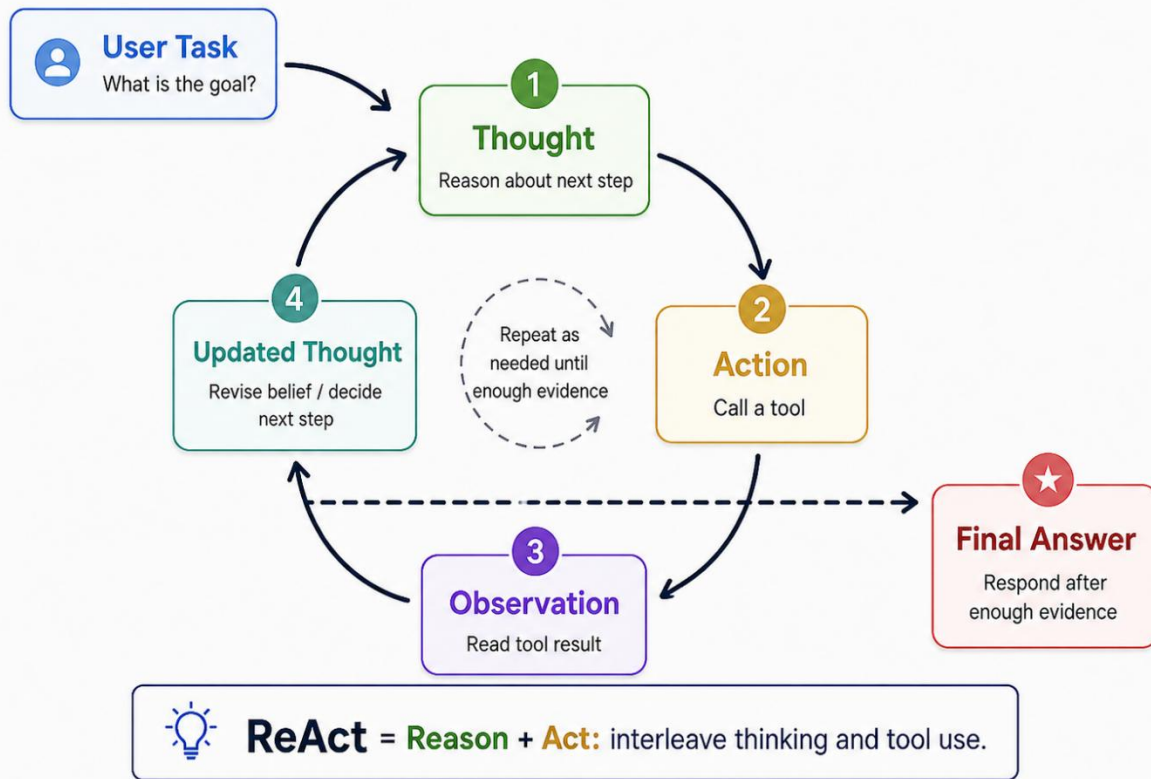


Agentic primitives

Reason + Act (ReAct)

The ReAct Loop for AI Agents

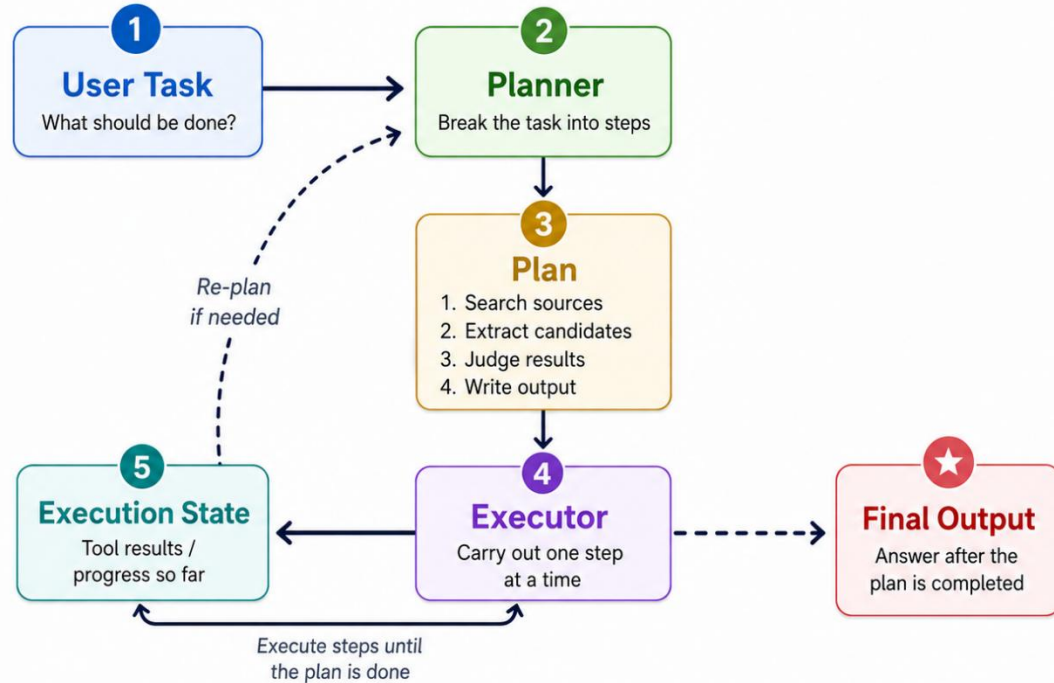
Iterate until sufficient evidence, then produce the final answer.



Planner-executor

The Planner–Executor Pattern for AI Agents

First make a high-level plan, then execute steps with tools.



Planner–Executor = Plan + Act: separate strategy from execution.

Agentic frameworks

What an agentic framework can do for you

- Automate tool execution + ingestion of tool call results
- Carry intermediate results (state) across multi-step runs
- Routing: decide which node/tool/step runs next
- Pause to get your approval
- Failure handling: retries, invalid outputs, failed tools, timeouts
- Tracing: seeing exactly what happened during the run

When an agentic framework may be relevant

- **Branching and iterative workflows:** gives you ready-made scaffolding
- **Failure-prone runs, expensive to rerun and stateful:** gives you checkpointing, retry/fallback routes, replay/debugging

Pros and cons of agentic workflows

Pros:

- Less manual coding
- Don't need to think whose APIs you use

Cons:

- Need to wrap your mind around a potentially very complex interface
- Ever-evolving product; bad documentation; potentially very painful when the product gets updated

Thank you for the being
the part of this course!